stalagmite project summary

Make stalagmites, not stalactites — a design-for-additive-manufacturing print-physics auditor for FDM/STL.

PUBLISHED · GitHub + PyPI | v0.7.0 | 37 tests passing | MIT license

Design intention

Slicers treat geometry as immutable: they paint overhangs red and answer every problem with support structures. Stalagmite exists to take the opposite stance — that the **design** is the thing to fix, not the scaffolding. Its purpose is prescriptive design feedback for parametric-CAD makers, the audience no existing tool serves: auto-orienters don't know a sealing face from a cosmetic one, and industrial repair suites are expert-operated and expensive.

Three intentions shaped every decision:

- **One physical rule, mechanically enforced.** Every slice must lie within a `dz·tan(45°)` dilation of the slice below. This single containment test subsumes overhang detection, floating features and unsupported starts — a geometric proxy for the print physics that a road only bonds where it lands on material beneath it.
- **Prescription over description.** It is not enough to flag a red region; the tool classifies *why* it fails and states a concrete fix with coordinates — “boss underside starts in air, hull it to the solid at (66, 0)”. Nothing in the hobbyist ecosystem says that sentence.
- **The transition is the shape.** The guiding design principle: never stack a wide feature on a narrow one at 90°; morph the junction (hull, cone, gusset) so the transition itself satisfies the physics. Supports are a last resort, not the answer.

Origin & validation basis

The tool was born from a real build: a threaded pH-probe holder iterated v2→v7 under a human slicer-eye review loop. Every failure mode encountered was diagnosed, fixed, and **frozen as a test fixture**. Those six STLs (worst → clean) are the regression suite — a genuine failure history rather than synthetic tests. The tool famously caught a 0.65 mm internal micro-bridge its own author had introduced, before any filament moved. The strongest validation of the repair engine is that, run on those fixtures, it independently re-prescribes the exact fixes the author discovered by trial and error — recommending the 45° morph cone and the grounded gusset that became versions 4 and 3.

What was built

Tier 1 — Audit

World-space slice-containment check (never per-slice 2-D frames, which hallucinate violations on asymmetric parts); severity-coloured PLY export for any mesh viewer.

Tier 2 — Classify

Every violation typed by in-plane anchoring — *starts-in-air, island, steep-growth, bridge* — and graded *fail / judge / tolerable* against published thresholds. Per-slice hits aggregate into physical defect features.

Tier 3 — Repair (`--suggest`)

Each failing feature gets parametrised fixes with real coordinates found by a 45° reachability search: ground/hull/gusset, morph the transition, teardrop/diamond, or reorient.

Tier 4 — Orient (`stalagmite-orient`)

Bayesian pose search (numpy Gaussian-process, ~35 evals) minimising a support proxy *subject to functional-surface constraints* — thread vertical, seal face as floor — which plain auto-orienters cannot express.

Helix auto-detect (`--auto-ex`)

Recognises thread helices by their signature (constant-area lobes circling at a fixed per-layer angle) and excludes them automatically — no manual zones. Reproduces the hand-tuned baseline exactly.

Interactive 3-D report (`--report`)

One self-contained, offline HTML file: rotate the part, click a defect to fly the camera to it, read the repair alongside. three.js is vendored inline so it never breaks.

Outcomes

- **Shipped and public.** Live on GitHub (public, MIT) and installable worldwide via `pip install stalagmite`; the PyPI/README page renders with the interactive-report screenshot.
- **Proven correct.** 37 automated tests, all passing, pinned to the six-fixture failure-history baseline; any future change must reproduce it.
- **Grounded, not guessed.** Thresholds trace to 17 studied DfAM sources (~1,125 pp). The 1.8 mm ledge cliff is Adam & Zimmer (2014); the 10 mm bridge span is Hinchy; the orientation method follows Goguelin (2021). The core rule was found to match Langelaar's additive-manufacturing filter from the topology-optimisation literature — independent convergence on the same formulation.
- **Field-tested.** First real-world part (`ada4965_sensor_holder`) audited cleanly end-to-end from the published install: one genuine fail (a 3.8 mm boss-underside overhang) with a correct morph/gusset prescription, plus a judged bridge and two tolerable ledges.

Key design decisions

- **Thresholds are parameters, not constants.** Every published number is boundary-condition-specific, so all are exposed as flags (`--angle` , `--dz` , `--min-wall` …) with community defaults — the tool adapts to the printer, not the reverse.
- **Human kept in the loop.** Small unsupported spans are reported as bridges for judgment, never auto-failed; a 5 mm round-hole roof is deliberately kept for tapping integrity. Prescription assists the maker; it doesn't overrule them.
- **Repairs must be re-audited.** The literature's own rules show a fillet can create a new overhang, so every suggestion carries a re-audit reminder — the loop is honest about its own limits.

Known limitations

The audit reflects the part *as oriented in the STL* — reorientation (Tier 4) is a separate step. The orientation proxy is facet-based and does not detect islands per candidate pose, so the chosen pose should be re-audited. Bridge free-span is measured on merged geometry as a roofed width, which is an estimate, not an exact hole diameter. Thresholds default to one material/machine profile and should be tuned per printer.

Future work

CadQuery integration example — show the audit inside a parametric workflow for the target audience.

Per-pose auditing in the orientation solver — fold the full Tier-2 classifier into orientation scoring.

Printer-profile presets — shippable JSON profiles so thresholds travel with the machine.

Broader field testing — run against a wider library of real parts; add a logo asset and gallery.

At a glance

Install	`pip install stalagmite`
Commands	`stalagmite` (audit) · `stalagmite-orient` (orientation)
Repository / package	github.com/Imagican/stalagmite · pypi.org/project/stalagmite
Dependencies	numpy, trimesh, shapely, networkx, scipy (auto-installed)
Update release	bump version in `pyproject.toml` → `python -m build` → `twine upload dist/*`

```
stalagmite "part.stl" --auto-ex --suggest --report "report.html"
stalagmite-orient "part.stl" --axis-vertical 0,0,1 --save "oriented.stl"
```


Generated 19 July 2026 · MIT licensed